CareerPilot

Dependencies & Documentation Report

This report inventories the dependencies used by CareerPilot, explains how they support the application's features, and serves as a project documentation guide so the repository can be reviewed, audited, and exported without needing to inspect every source file first.

---

1. Dependency Inventory

1.1 Frontend Dependencies

Library	Purpose
next	App Router, route-based layouts, server/client rendering. Page structure, shared layout, and protected UI flow are all built on Next.js.
react / react-dom	Power the entire interface layer: dashboard, chat, tracker, profile upload, and CV builder.
typescript	Type safety for API contracts, component props, and state shapes — critical because the frontend relies heavily on typed backend responses.
lucide-react	Iconography across the dashboard, tracker, chat, profile, and navigation UI.
tailwindcss / @tailwindcss/postcss	Styling pipeline. Works alongside the project's custom CSS system.
eslint / eslint-config-next	Linting and maintaining frontend code quality.

1.2 Backend Dependencies

Library	Purpose
fastapi	Primary backend framework — defines the HTTP API surface for auth, CV upload, chat, jobs, tracker, and document generation.
uvicorn[standard]	Runs the FastAPI app with production-friendly ASGI support.
python-multipart	Required for file upload handling, especially the CV ingestion endpoint.

Library	Purpose
langchain langchain-core langchain-community langchain-google-genai	Assistant orchestration layer, chat memory interfaces, prompt construction, and Google Gemini integration.
redis / redis-py	Caching layer for job search results, fit scores, and session tokens. Reduces repeated LLM and database calls, and supports fast token invalidation on logout.
chromadb	Stores and retrieves embedded CV chunks for RAG-style question answering.
sentence-transformers	Local embedding model used to transform CV text into vectors.
pypdf	Extracts text from uploaded PDF CVs.
python-docx	Reads uploaded DOCX CVs and generates editable DOCX output in the CV builder.
sqlalchemy	ORM models and database sessions for users, chat messages, applications, todos, and activities.
psycopg2-binary	PostgreSQL driver for SQLAlchemy.
python-jose[cryptography]	JWT creation and verification.
passlib[bcrypt] / bcrypt	Secure password hashing and verification.
reportlab	Generates the PDF version of the AI-created CV.
pydantic-settings	Loads configuration from environment variables and validates required settings.
python-dotenv	Supports local development by loading .env values.
httpx	Async HTTP requests to external job-search services.
tenacity	Retry logic for rate-limited LLM requests.
python-dateutil	Date handling utilities used across the backend.
google-api-core	Gemini/Google API exception handling, especially rate-limit detection.

---

2. Why These Dependencies Exist

The dependency set is intentionally practical rather than excessive. Each library supports a concrete product capability. Auth and persistence use standard, proven Python libraries. Retrieval and embeddings are local where possible to reduce cost. **Redis provides a fast caching layer that prevents redundant job searches, repeated fit score computations, and unnecessary database hits** — keeping response times low under real usage. Document generation uses specialized tooling instead of a homemade formatter. External job search is isolated behind HTTP client calls so it can fail gracefully. Retry and memory behavior are implemented to support production-like reliability.

---

3. Feature-to-Dependency Mapping

Feature	Libraries Used
Authentication	FastAPI · SQLAlchemy · python-jose · passlib / bcrypt · Redis (token cache & invalidation)
CV Upload & Analysis	FastAPI multipart · pypdf · python-docx · LangChain text splitting · sentence-transformers · ChromaDB
Career Chat	LangChain · Gemini (google-genai) · PostgreSQL-backed memory · tenacity · Redis (session caching)
Job Search	httpx · fuzzy matching · deterministic fit scoring · Redis (result caching)
Application Tracking	SQLAlchemy models · frontend drag-and-drop board
CV Builder	ReportLab (PDF) · python-docx (DOCX)

---

4. Documentation Map

4.1 Core Repository Documents

Document	Description
README.md	Main project overview: repository structure, features, stack, quick start, and custom tests.
walkthrough.md	Project narrative and presentation aid. Use when demonstrating the user journey or demo flow.
frontend/README.md	Default Next.js starter guidance and frontend entry-point commands.
frontend/AGENTS.md	Workspace-specific guidance for the frontend area.
frontend/CLAUDE.md	Additional frontend guidance and conventions.

4.2 Backend Documentation Surfaces

File	Description
backend/app/main.py	App startup, router registration, CORS policy, and health endpoint.
backend/app/config.py	Required environment variables and derived data paths.
backend/app/models/db_models.py	Database entities that power the product.
backend/app/models/schemas.py	Request and response contracts for the API.
backend/app/services/*.py	Core business logic: CV processing, job matching, fit scoring, memory, and generation.
backend/app/cache.py	Redis client setup and helper functions for reading and writing cached results.

---

5. Configuration & Environment Variables

- GOOGLE_API_KEY
- LLM_MODEL
- EMBEDDING_MODEL
- DATABASE_URL
- JWT_SECRET_KEY
- REDIS_URL *(e.g. redis://localhost:6379/0)*
- SERPAPI_API_KEY (if live job search enabled)

The backend also creates data directories for Chroma persistence and uploaded files. Those directories are important because the app relies on persisted vector data and file storage, not only API responses. Redis must be reachable at startup — if the REDIS_URL is missing or unreachable the caching layer degrades gracefully but performance will be reduced.

---

6. Suggested Review Order

- **1.** Root README.md — product overview
- **2.** backend/app/main.py — application wiring
- **3.** backend/app/models/db_models.py — data persistence
- **4.** backend/app/cache.py — Redis caching setup
- **5.** backend/app/services/cv_processor.py — CV ingestion
- **6.** backend/app/services/agent.py — assistant behavior
- **7.** frontend/src/lib/api.ts — frontend-backend integration
- **8.** frontend/src/app/page.tsx, chat/page.tsx, profile/page.tsx, tracker/page.tsx — user-facing flows

---

7. Operational Notes

- The app is designed to run with Docker Compose — the simplest way to start the full stack.
- The backend expects working database and API credentials for the full experience.
- Redis must be running and reachable for caching to function. Docker Compose starts it automatically alongside the database and backend.
- Some features have graceful fallbacks, such as mock job data when external job APIs are unavailable.
- The chat system is resilient to rate limiting, but Gemini quotas still matter for real deployments.

Summary: The project's dependencies are well aligned with the product goals. The stack is not inflated with unrelated libraries. Instead, the repository uses a focused set of tools for API delivery, persistence, caching, retrieval, AI orchestration, file processing, and document generation.